

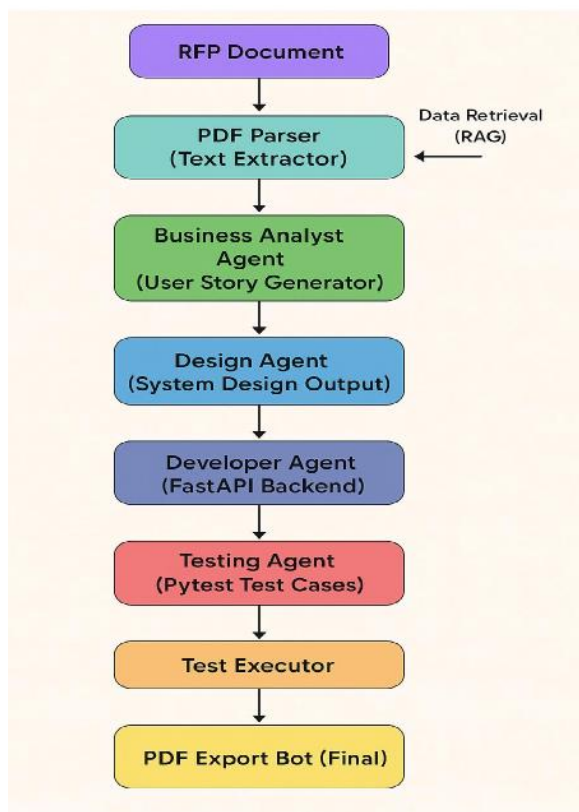
Introduction

This project introduces the AI-Powered Virtual Development Pod, a cutting-edge system designed to simulate the complete software development lifecycle (SDLC) using AI agents and a Retrieval-Augmented Generation (RAG)-inspired Large Language Model (LLM) architecture.

The system automates the end-to-end transition from business requirements—provided in the form of a Request for Proposal (RFP)—to fully deployable backend code and corresponding test cases. It achieves this by leveraging conversational AI agents, each representing critical roles in a typical development team, including the Business Analyst, System Designer, Developer, and Tester.

By integrating LLMs with RAG, the platform creates a dynamic virtual software development environment capable of autonomously generating user stories, detailed system design documents, functional backend code, and comprehensive test cases. This approach streamlines the SDLC, enhances productivity, and reduces the time and effort traditionally required for software delivery.

Flow Diagram



Project Architecture

Component	Technology / Tool	Description
UI / Frontend	Gradio	Python-based UI library for building interactive web interfaces
LLM API	OpenRouter	API gateway for large language models; uses Mixtral 8x7B for high-quality responses
PDF Parsing	PyMuPDF	Lightweight and fast PDF parser to extract structured text from RFP documents
Backend Framework	FastAPI	Modern Python web framework used for the generated backend code
Testing Framework	Pytest	Testing suite for automated validation of generated FastAPI backend
Exporting	FPDF	PDF generation tool to compile and export final output artifacts
Orchestration	Python Functions	Manages the end-to-end agent flow and logic chaining between components
Agent Flow Memory	Python Dictionary (session_state)	Maintains stateful information across the agent pipeline during runtime

Agents' Role

This GenAI pipeline is composed of modular agents, each designed to perform a specialized role within the Software Development Life Cycle (SDLC). The agents work in a synchronized sequence, where the output of one serves as the input to the next, ensuring seamless automation from requirement analysis to final delivery.

Business Analyst Agent

- Role: Translates the RFP document into structured functional requirements.
- Prompt Task: Extract and generate 10 user stories from the provided RFP text.
- Output: Well-structured user stories with titles, descriptions, and acceptance criteria.
- Importance: Drives the entire downstream pipeline by forming the basis for design, development, and testing.

Design Agent

- Role: Converts user stories into high-level technical design.

- Dependencies: Requires input from the Business Analyst Agent.
- Output Includes:
 - o System architecture overview (e.g., microservices, layers)
 - o Database schema (ERD or table design)
 - o API structure and endpoint blueprint
- Purpose: Acts as a bridge between requirements and code generation.

Developer Agent

- Role: Generates production-ready backend code using FastAPI.
- Dependencies: Consumes user stories and system design documents.
- Output:
 - o FastAPI application with defined routes
 - o Data models (e.g., Pydantic models)
 - o Business logic and validation rules
- Goal: Automate backend creation that adheres to the provided specs.

Testing Agent

- Role: Develops automated unit and integration test cases.
- Dependencies: Requires both the user stories and generated backend code.
- Output:
 - o Pytest test scripts aligned with each endpoint and logic path
 - o Test cases for edge handling and expected behavior
- Benefit: Validates correctness, robustness, and edge scenarios.

Test Runner Agent

- Role: Executes all generated test cases.
- Dependencies: Needs both the FastAPI backend and Pytest scripts.
- Output:
 - o Real-time test execution results (pass/fail logs)
 - o Summary of test coverage and error handling
- Utility: Ensures quality assurance before deployment.

Exporting Agent

- Role: Finalizes and compiles all outputs into a single deliverable.
- Dependencies: Aggregates content from all agents.
- Output: A well-formatted PDF report containing:
 - o User Stories
 - o Design Artifacts
 - o Generated Code Summary
 - o Test Cases & Execution Logs
- Value: Delivers a complete SDLC trace in a client-ready format

Agent Synchronization Overview

The agents follow a sequential pipeline, each depending on the output of the previous stage. They communicate and pass data through a shared session state (a Python dictionary), ensuring smooth transitions and contextual continuity.

1. User uploads RFP(Request For Proposal) → Business Analyst Agent runs → Business Analyst Agent output (User Stories).
2. Business Analyst output (User Stories) → Design Agent runs → Design Agent output (Design Document).
3. Business Analyst output (User Stories) + Design Agent output (Design Document) → Developer Agent runs → Developer Agent Output (Backend Code).
4. Business Analyst output (User Stories) → Testing Agent runs → Testing Agent output (Test Cases)
5. Developer Agent output (Backend Code) + Testing Agent output (Test Cases) → Test Runner runs → Test Runner output (Test cases validation).
6. Business Analyst output (User Stories) + Design Agent output (Design Document) + Developer Agent output (Backend Code) + Testing Agent output (Test Cases) → Export Bot compiles into PDF.

Conclusion

The AI-powered Virtual Development Pod represents a transformative leap in intelligent automation within the realm of software engineering. By integrating Large Language Models (LLMs) with a Retrieval-Augmented Generation (RAG)-inspired architecture, the system achieves dynamic responsiveness to real-world documents such as RFPs.

This architecture enables the seamless automation of traditionally manual and time intensive stages across the Software Development Life Cycle (SDLC) — from requirements gathering and system design, to code generation, testing, and final delivery. Through modular agents synchronized via shared memory, this solution brings speed, scalability, and consistency to software development pipelines — paving the way for next gen AI-driven engineering teams.